

PocketSmart AI

File 5 — Project Development

Project: GenAI-Powered Budget & Recommendation Assistant

This document describes the development stage of PocketSmart AI, including the implementation approach, major software modules, user flow, AI recommendation workflow, data handling, validation, and deployment preparation.

1. Development Overview

The development phase converts the approved requirements and project design into a working application. PocketSmart AI is planned as a web-based recommendation assistant that accepts a user's budget and requirements, processes those inputs, sends an appropriate prompt to a generative AI model, and presents structured recommendations.

2. Main Development Modules

Module	Development Responsibility
User Input Module	Collect category, budget, preferences, requirements, and other relevant constraints.
Input Validation	Check required fields, acceptable values, and budget-related inputs before processing.
AI Recommendation Module	Construct prompts and communicate with the Gemini API to generate recommendations.
Recommendation Formatter	Convert AI output into a readable and consistent presentation for the user.
User Interface	Provide forms, controls, recommendation cards/sections, navigation, and responsive layouts.
Backend / API Layer	Handle application logic, requests, validation, AI service communication, and errors.
Data Layer	Store application data when persistence is required by the implemented version.
Error Handling	Display useful messages for invalid inputs, API failures, unavailable services, and unexpected responses.

3. User-to-Recommendation Workflow

- Step 1:** User opens PocketSmart AI.
- Step 2:** User selects or enters the required category.
- Step 3:** User provides a budget and relevant preferences.
- Step 4:** The application validates the submitted information.
- Step 5:** The backend prepares a structured AI prompt.
- Step 6:** The Gemini API processes the request.
- Step 7:** The application receives the generated recommendation content.
- Step 8:** The response is validated and formatted.
- Step 9:** The user views recommendations according to the requested budget and requirements.

4. AI Integration

The AI layer is responsible for converting user requirements into useful recommendation content. The application should provide the model with explicit information about the user's budget, category, preferences, and constraints so that the generated response remains aligned with the request.

Example prompt structure:

```
Category: {category}  
Budget: {budget}  
Preferences: {preferences}  
Requirements: {requirements}
```

Generate practical recommendations that respect the stated budget and clearly explain the reasoning or relevant details.

5. Frontend Development

- Create a clean landing/input interface for entering recommendation requirements.
- Use clearly labelled fields for category, budget, preferences, and additional requirements.
- Provide a visible action control for generating recommendations.
- Show loading/progress feedback while the AI request is being processed.
- Present results in a structured format that is easy to scan.
- Handle empty, invalid, or incomplete inputs with clear validation messages.
- Maintain responsive behavior across common desktop and mobile screen sizes.

6. Backend Development

- Receive and validate user input.
- Prepare the AI request using a controlled prompt format.
- Authenticate with the configured Gemini API credentials.
- Send the request to the selected Gemini model.
- Handle API errors, quota limitations, timeouts, and unavailable-model responses.
- Return a structured response to the frontend.
- Keep secret API credentials in environment variables rather than hard-coding them in source code.

7. Recommendation Processing

Recommendation processing should preserve the user's constraints. In particular, the generated content should be checked for obvious inconsistencies with the requested budget or category before being displayed. Where exact product availability or pricing is not connected to a live product data source, the application should present the AI output as recommendations rather than guaranteeing current availability or exact prices.

8. Data Handling

Data	Purpose
Category	Determines the type of recommendation requested.
Budget	Defines the user's spending constraint.
Preferences	Personalizes the recommendation criteria.
Requirements	Captures additional user constraints or desired features.
Generated Response	Contains the recommendation output returned by the AI service.

9. Error Handling

Situation	Application Response
Missing required input	Ask the user to complete the required field.
Invalid budget	Display a validation message and request a valid value.
Gemini API failure	Show a user-friendly error and avoid exposing secret/API details.
Quota or rate limit	Inform the user that the AI service is temporarily unavailable and allow retrying later.
Unexpected AI response	Use safe fallback formatting or request regeneration rather than crashing.
Network/deployment failure	Return an appropriate error response and log the technical details for debugging.

10. Development Output

- Working PocketSmart AI user interface.
- Functional user-input and validation flow.
- Gemini-powered recommendation generation.
- Formatted recommendation results.

- Error handling for common application and AI-service failures.
- Source code organized into maintainable frontend/backend modules.
- Deployment-ready environment configuration.

11. Development Completion Criteria

The development phase can be considered ready for testing when the primary user journey—from entering a budget and requirements through receiving a formatted recommendation—works end-to-end, validation is active, major failure conditions are handled, and the application can be run using the documented setup process.